PRACTICAL - 8

Implementation of Graph Searching using DFS and BFS

Aim of the Practical:

To implement Breadth First Search (BFS) and Depth First Search (DFS) for graph traversal using Python and analyze their time and space complexities.

Theory:

A graph is represented by an adjacency list. BFS explores vertices level by level using a queue. DFS follows one path as deeply as possible using a stack and then backtracks. A visited set prevents a vertex from being processed more than once.

Algorithm:

BFS: add the start vertex to the queue and visited set; repeatedly remove the front vertex, record it, and enqueue each unvisited neighbor. DFS: push the start vertex; repeatedly pop the top vertex, and if it is unvisited, record it and push its unvisited neighbors in reverse order so their original order is preserved during traversal.

Program (Python):

```
def bfs(graph, startnode):
    visited = set()
    queue = [startnode]
    traversal_order = []

    visited.add(startnode)

    while queue:
        current_node = queue.pop(0)
        traversal_order.append(current_node)

        for neighbor in graph[current_node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return traversal_order


def dfs(graph, startnode):
    visited = set()
    stack = [startnode]
    traversal_order = []

    while stack:
        current_node = stack.pop()

        if current_node not in visited:
            visited.add(current_node)
            traversal_order.append(current_node)

            for neighbor in reversed(graph[current_node]):
                if neighbor not in visited:
                    stack.append(neighbor)

    return traversal_order


graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

print("BFS:", bfs(graph, 'A'))
```

```
print("DFS:", dfs(graph, 'A'))
```

Output and Traversal Traces:

```
BFS: ['A', 'B', 'C', 'D', 'E', 'F']
DFS: ['A', 'B', 'D', 'E', 'F', 'C']

BFS QUEUE TRACE
Step  Processed  Queue after processing  Traversal
0     -          [A]                     []
1     A          [B, C]                  [A]
2     B          [C, D, E]               [A, B]
3     C          [D, E, F]               [A, B, C]
4     D          [E, F]                  [A, B, C, D]
5     E          [F]                     [A, B, C, D, E]
6     F          []                      [A, B, C, D, E, F]

DFS STACK TRACE (top of stack is shown at the right)
Step  Popped  Stack after processing  Traversal
0     -       [A]                    []
1     A       [C, B]                 [A]
2     B       [C, E, D]              [A, B]
3     D       [C, E]                 [A, B, D]
4     E       [C, F]                 [A, B, D, E]
5     F       [C, C]                 [A, B, D, E, F]
6     C       [C]                    [A, B, D, E, F, C]
7     C       []                     already visited; skipped

FINAL RESULTS
BFS traversal: A -> B -> C -> D -> E -> F
DFS traversal: A -> B -> D -> E -> F -> C
BFS time complexity: Theta(V + E)
DFS time complexity: Theta(V + E)
BFS auxiliary space: Theta(V)
DFS auxiliary space: Theta(V)
```

BFS Queue Table:

Step	Processed	Queue
0	-	[A]
1	A	[B,C]
2	B	[C,D,E]
3	C	[D,E,F]
4	D	[E,F]
5	E	[F]
6	F	[]

DFS Stack Table (top at right):

Step	Popped	Stack
0	-	[A]
1	A	[C,B]
2	B	[C,E,D]
3	D	[C,E]
4	E	[C,F]
5	F	[C,C]
6	C	[C]
7	C	[] - skipped

Table Interpretation:

BFS processes A, then its level-one neighbors B and C, followed by D, E, and F. DFS uses reversed neighbor insertion so B is visited before C. Vertex C appears twice in the stack because it was already waiting when F examined C; the second copy is safely skipped by the visited check.

Time Analysis:

Let V be the number of vertices and E the number of edges. Each vertex is processed once, giving Theta(V). Across all adjacency lists, all neighbor scans require Theta(E). Therefore both traversals take Theta(V + E).

Space Analysis:

The visited set, traversal list, and queue or stack can each contain at most V vertices. Thus BFS and DFS both require Theta(V) auxiliary space.

Final Analysis:

- BFS Time: Theta(V + E); BFS Space: Theta(V).

- DFS Time: Theta(V + E); DFS Space: Theta(V).

Result:

BFS and DFS were implemented successfully in Python. Starting at A, BFS produces A, B, C, D, E, F and DFS produces A, B, D, E, F, C.